

Hướng dẫn đồ án (working notes)

Ghi chú phân tầng nguồn: đây là *sổ tay làm việc* (working notes) từ giai đoạn khảo sát — chứa cả nguồn cấp 1 (tài liệu chính chủ, repo gốc) lẫn cấp 2 (blog/bài hướng dẫn — dùng định hướng, KHÔNG trích vào báo cáo). Danh mục nguồn *chính thức* = 9 bibitem trong docs/report/main.tex, toàn bộ đã kiểm sống.

Hướng dẫn triển khai toàn bộ đồ án NT219 — Benchmark Kyber/Dilithium vs RSA/ECC

Tài liệu này chia đồ án thành các **Work Package (WP)**, mỗi WP gồm nhiều **task nhỏ**. Mỗi task quan trọng trình bày **HAI CÁCH**: (A) dùng lệnh/công cụ có sẵn — nhanh, đỡ sai; (B) tự thiết lập — hiểu sâu, chủ động. Mỗi nguồn đều kèm **link kiểm chứng**. Các điểm dễ gây “crypto failure” được đánh dấu [!].

Bản đồ tổng thể (ánh xạ với đề bài thầy)

WP	Tên	Mục đề bài	Câu hỏi nghiên cứu
WP1	Dựng môi trường (OpenSSL 3.6 + liboqs)	§7.2, §7.3, §8	nền cho tất cả
WP2	Microbenchmark (key-gen/encap/decap/sign/verify)	§7.4, §9	RQ1 (compute)
WP3	NEON + ARM (Raspberry Pi / cloud ARM)	§7.2, §8.1	RQ2
WP4	TLS end-to-end (hand-shake, throughput)	§7.6, §9	RQ3
WP5	Tài nguyên tĩnh (kích thước khóa/sig, RSS, code size)	§9	RQ1 (bandwidth), RQ2
WP6	Năng lượng (ước lượng)	§9, §13	phụ

WP	Tên	Mục đề bài	Câu hỏi nghiên cứu
WP7	Thống kê + trả lời RQ + đóng gói	§7.5, §11, §12	tổng hợp

Thứ tự ưu tiên theo rubric (Rigor 30% + Reproducibility 25% + Analysis 15% = 70%): WP1 → WP2 → WP5 → WP4 → WP3 → WP6 → WP7.

Sự thật nền tảng đã kiểm chứng (đọc trước khi làm)

OpenSSL ≥ 3.5 có sẵn ML-KEM/ML-DSA/SLH-DPA trong provider mặc định — không cần fork, không cần oqs-provider cho phần OpenSSL thuần. Bản chính thức nêu rõ ba thuật toán: ML-KEM (FIPS 203), ML-DSA (FIPS 204), SLH-DPA (FIPS 205). - Link: <https://openssl-library.org/post/2025-02-04-release-announcement-3.5/> - Link (3.5 final, LTS tới 2030): <https://openssl-foundation.org/post/2025-04-08-openssl-35-final-release/>

Đây là điểm cốt lõi định hình đồ án: bạn có **hai con đường** để có PQC trong OpenSSL, và phải hiểu rõ để không lẫn (lẫn là sinh “crypto failure”): - **Con đường 1 — OpenSSL native (≥ 3.5)**: PQC nằm thẳng trong libcrypto. Dùng cho phần lớn đồ án. Tên thuật toán: ML-KEM-768, ML-DSA-65; nhóm TLS: mlkem768, X25519MLKEM768. - **Con đường 2 — liboqs + oqs-provider**: thư viện riêng (liboqs) cắm vào OpenSSL qua provider. Cần cho WP3 vì liboqs có công tắc NEON tường minh (OQS_USE_ARM_NEON_INSTRUCTIONS) và đếm chu kỳ ARM (OQS_SPEED_USE_ARM_PMU) mà OpenSSL native không có.

[!] **Nguyên tắc số 1 chống crypto failure**: đừng trộn hai con đường trong một phép đo. Hoặc đo bằng OpenSSL native, hoặc đo bằng liboqs — ghi rõ trong báo cáo dùng cái nào. Trộn (ví dụ link libcrypto native nhưng nạp oqs-provider) để gây xung đột tên thuật toán và sai số.

WP1 — Dụng môi trường build

Mục tiêu: có openssl 3.6.x chạy được, có ML-KEM/ML-DSA, **không** dụng OpenSSL hệ thống (tránh link nhầm → crypto failure).

Task 1.1 — Cài công cụ build

Trên Ubuntu/Debian (x86 và Pi đều vậy):

```
sudo apt-get update
sudo apt-get install -y build-essential git cmake ninja-build perl pkg-config
# cho phần đo sau này:
sudo apt-get install -y time tshark
```

- build-essential = gcc/g++/make; perl là BẮT BUỘC để cấu hình OpenSSL.
- Nguồn yêu cầu công cụ liboqs (cmake, ninja): <https://openquantumsafe.org/liboqs/getting-started.html>

Task 1.2 — Build OpenSSL 3.6 (có PQC native)

Đây là task xương sống. Trình bày hai cách.

CÁCH A — Dùng script có sẵn của pqcbench (lelegard)

Repo lelegard/pqcbench có sẵn rebuild-openssl.sh tải nguồn, biên dịch OpenSSL 3.6, rồi setenv-rebuilt-openssl.sh để nạp môi trường. Quy trình chính chủ:

```
git clone https://github.com/lelegard/pqcbench.git
cd pqcbench
./rebuild-openssl.sh
source setenv-rebuilt-openssl.sh
make build/pqcbench
```

- Trích README: To download OpenSSL sources, recompile OpenSSL 3.6, build and run the pqcbench test, run the following commands: ./rebuild-openssl.sh ; source setenv-rebuilt-openssl.sh ; make build/pqcbench. README cũng nhấn mạnh script setenv **phải được source**, không chạy.
- Link: <https://github.com/lelegard/pqcbench>
- [!] Vì sao phải source: script đặt PATH/LD_LIBRARY_PATH cho shell hiện tại. Nếu chạy (./setenv...) thì biến chỉ sống trong tiến trình con rồi mất → gõ openssl vẫn ra bản hệ thống (không PQC) → crypto failure. (Cùng lý do cha/con với setenv.sh của bạn.)

CÁCH B — Tự cấu hình OpenSSL từ nguồn (kiểm soát hoàn toàn)

Dựa trên INSTALL.md chính thức của OpenSSL. Cách này cho bạn quyết định đường cài (đề vào \$HOME, không cần sudo, không đụng hệ thống):

```
# 1. Lấy nguồn theo tag ổn định
git clone https://github.com/openssl/openssl.git
cd openssl
git checkout openssl-3.6.2 # tag phát hành; ghi lại commit để tải

# 2. Cấu hình: cài vào $HOME/pqc/openssl (KHÔNG /usr, KHÔNG sudo)
./Configure --prefix="$HOME/pqc/openssl" --openssldir="$HOME/pqc/openssl/ssl"

# 3. Biên dịch + tự kiểm tra + cài
make -j"$(nproc)"
make test # nên chạy: xác nhận build lành mạnh
make install_sw install_ssldirs # install_sw: chỉ phần mềm, bỏ man page
```

- Link INSTALL chính thức: <https://github.com/openssl/openssl/blob/master/INSTALL.md>
- [!] **Điểm chống crypto failure quan trọng nhất của cả đồ án:** cài vào \$HOME/pqc/openssl (prefix riêng) để KHÔNG đề lên /usr/bin/openssl của hệ điều hành. Nếu cài đề hệ thống, các chương trình khác (apt, ssh) có thể gây, và bạn khó biết đang dùng bản nào. Prefix riêng = sạch, gỡ chỉ cần xóa thư mục.
- [!] Sau khi cài, để dùng bản này phải đặt LD_LIBRARY_PATH trỏ tới lib/lib64 của nó (xem Task 1.4). Quên bước này → chạy nhị phân báo libssl.so.3: cannot open shared object hoặc tệ hơn là âm thầm nạp libcrypto hệ thống (thiếu PQC).

Xác minh (BẮT BUỘC, cả hai cách)

```
"$HOME/pqc/openssl/bin/openssl" version # phải in 3.6.x
"$HOME/pqc/openssl/bin/openssl" list -kem-algorithms | grep -i mlkem
"$HOME/pqc/openssl/bin/openssl" list -signature-algorithms | grep -i ml-dsa
```

Thấy ML-KEM-768, ML-DSA-65... là PQC native đã sẵn. RHEL lab chính thức xác nhận openssl list liệt kê MLKEM512/768/1024 và các hybrid như X25519MLKEM768. - Link: <https://www.cryptomathic.com/blog/quantum-ready-cryptography-with-openssl-3.5-on-rhel-9.6>

Task 1.3 — (Tùy WP3) Build liboqs

Chỉ cần nếu làm WP3 (NEON/ARM cycle). Trên x86 build trước cho quen, rồi mới sang ARM.

CÁCH A — fullbuild script của oqs-provider

oqs-provider có scripts/fullbuild.sh tự lo cả liboqs lẫn provider:

```
git clone https://github.com/open-quantum-safe/oqs-provider.git
cd oqs-provider
./scripts/fullbuild.sh
```

- Link: <https://github.com/open-quantum-safe/oqs-provider>
- Bài Santander mô tả luồng này (clone oqs-provider → ./scripts/fullbuild.sh → cmake install): <https://medium.com/be-tech-with-santander/how-to-configure-post-quantum-cryptography-in-your-web-server-fcf79e05e526>

CÁCH B — Tự cmake liboqs (kiểm soát cờ build, cần cho WP3)

Theo wiki cấu hình chính thức của liboqs:

```
git clone https://github.com/open-quantum-safe/liboqs.git
cd liboqs
git checkout 0.14.0 # tag ổn định
cmake -GNinja -S . -B build \
  -DCMAKE_INSTALL_PREFIX="$HOME/pqc/liboqs" \
  -DBUILD_SHARED_LIBS=ON \
  -DOQS_USE_OPENSSL=ON \
  -DOPENSSL_ROOT_DIR="$HOME/pqc/openssl"
ninja -C build
ninja -C build install
```

- Mặc định liboqs build **static** (liboqs.a); thêm -DBUILD_SHARED_LIBS=ON để có liboqs.so. Nguồn: <https://openquantumsafe.org/liboqs/getting-started.html>
- Các tùy chọn build: <https://openquantumsafe.org/liboqs/api/doxygen/options-for-configuring-liboqs-builds.html>
- [!] -DOPENSSL_ROOT_DIR phải trỏ tới OpenSSL bạn vừa build, để liboqs dùng AES/SHA của nó nhất quán — tránh trộn hai OpenSSL.
- (Phần NEON/PMU và “hai cây ref/neon” để ở WP3.)

Task 1.4 — Quản lý phiên bản + môi trường (tái lập = 25% điểm)

[!] Đây là phần ăn điểm **Reproducibility**. Ý tưởng: gom mọi phiên bản/đường dẫn vào MỘT chỗ (versions.env), và có setenv.sh để nạp môi trường.

scripts/versions.env (single source of truth) — pin tag để tái lập:

```
export PQC_ROOT="$HOME/pqc"
export OSSL_PREFIX="$PQC_ROOT/openssl"
export OPENSSL_TAG="openssl-3.6.2"
export LIBOQS_TAG="0.14.0"
```

scripts/setenv.sh (PHẢI source) — đặt PATH + LD_LIBRARY_PATH:

```
source "$ (dirname "${BASH_SOURCE[0]}") /versions.env"
export PATH="$OSSL_PREFIX/bin:$PATH"
export LD_LIBRARY_PATH="$OSSL_PREFIX/lib:$OSSL_PREFIX/lib64:${LD_LIBRARY_PATH}"
```

- [!] Việc dùng LD_LIBRARY_PATH (thay vì nhúng rpath) chính là quy ước của demo OpenSSL chính thức (LD_LIBRARY_PATH=../../demo). Xem openssl/demos/.../Makefile.

- Đề bài yêu cầu tái lập: ghi rõ phiên bản gcc/clang, cờ build, governor CPU (§7.3).

WP2 — Microbenchmark (đo thời gian từng thao tác)

Mục tiêu: đo keygen / encap-decap (KEM) / sign-verify (chữ ký) cô lập, N vòng, lấy median/mean/std/p95/p99/CI. Đây là mặt **compute** của RQ1.

Task 2.1 — Chọn cách đo: ba lựa chọn

CÁCH A1 — openssl speed (nhANH NHẤT, có sẵn)

OpenSSL 3.5+ đo PQC ngay bằng lệnh speed:

```
source scripts/setenv.sh
openssl speed -seconds 3 rsa2048 ecdsap256
openssl speed mlkem768 # nếu build hỗ trợ speed cho KEM
```

- Ảnh bạn gửi chính là cái này: `openssl speed rsa2048` → cho sign/s, verify/s. Đây là baseline tin cậy để đối chiếu.
- OQS profiling cũng dùng `openssl speed` ở tầng ứng dụng: <https://github.com/open-quantum-safe/profiling>
- [!] Hạn chế: `openssl speed` đo theo kiểu “bao nhiêu op trong X giây”, khó lấy phân vị p95/p99 và CI như đề bài §7.4 đòi. Dùng nó làm *đối chứng*, không làm dữ liệu chính.

CÁCH A2 — liboqs speed_kem / speed_sig (chuẩn học thuật)

liboqs có sẵn hai chương trình benchmark: - Trích README liboqs: `speed_kem`: Benchmarking program for key encapsulation mechanisms; `speed_sig`: Benchmarking program for signature mechanisms. - Link: <https://github.com/open-quantum-safe/liboqs>

```
./build/tests/speed_kem ML-KEM-768
./build/tests/speed_sig ML-DSA-65
```

- Framework PQC-LEO dùng đúng hai công cụ này, chạy mỗi op trong cửa sổ thời gian cố định (mặc định 3 giây): https://github.com/crt26/PQC-LEO/blob/main/docs/performance_results/performance_metrics
- [!] `liboqs speed_*` đo *thuật toán liboqs*, không phải OpenSSL. Nếu báo cáo so “PQC vs RSA”, phần RSA phải lấy từ OpenSSL → hai nguồn khác nhau, nêu rõ trong Methodology.

CÁCH B — Harness EVP tự viết (bench_evp.cpp — KHUYẾN NGHỊ làm chính)

Đây là cách để bài ngầm yêu cầu (§8.3: “build microbenchmark harness that calls keygen/encaps/decaps/sign/verify in tight loops with high-resolution timers `clock_gettime CLOCK_MONOTONIC_RAW`”). Bạn đã có `bench_evp.cpp` làm đúng điều này: - Dùng `clock_gettime(CLOCK_MONOTONIC_RAW)` cho wall-clock — đúng nguồn yêu cầu. - Tự tính median/mean/std/p95/p99/CI — đáp ứng §7.4 mà `openssl speed` không cho. - Gọi qua EVP API thống nhất cho CẢ RSA/EC/PQC → so sánh công bằng trên cùng một lớp đo. - Ưu điểm lớn nhất: cùng một công cụ đo mọi thuật toán = loại bỏ sai khác do công cụ.

[!] **Vì sao tự viết tốt hơn cho đồ án này:** đề bài chấm Rigor 30% dựa trên median-of-medians, bootstrap CI, paired comparison (§7.5). `openssl speed/liboqs speed_*` cho số tổng hợp sẵn, bạn không kiểm soát được cách lấy mẫu. Harness tự viết xuất mẫu thô → bạn tự làm thống kê đúng yêu cầu.

Task 2.2 — Tham số đo (theo §7.4 và §7.5)

- N iterations: 1000-10000 mỗi op (đề bài §7.4). Op nhanh (sign/verify) dùng N lớn; keygen chậm (RSA) dùng N nhỏ hơn.

- Warm-up: loại bỏ vài chục vòng đầu (cache/lazy-init) trước khi đo (§7.5).
- K batch × M iter: chạy 5-10 batch, lấy median-of-medians, CI bootstrap (§7.5).
- [!] Cố định tần số CPU (tắt turbo), pin core để giảm nhiễu (§7.4 “fix CPU frequency, isolate cores”):

```
sudo cpupower frequency-set -g performance # governor hiệu năng, giảm dao
```

Task 2.3 — Lưu kết quả

bench_evpc in ra stdout (CSV thô qua BENCH_CSV, tóm tắt key-value qua stdout). Lưu bằng redirect:

```
mkdir -p data/raw/$(uname -m)
BENCH_CSV=data/raw/$(uname -m)/mlkem768.raw.csv \
./build/bench_evpc mlkem 768 > data/raw/$(uname -m)/mlkem768.kv.txt
```

Script run_micro.sh đã tự động hóa chạy cả ma trận → CSV tổng.

WP3 — NEON + ARM (phần khó & rủi ro nhất, trả lời RQ2)

Mục tiêu: chứng minh tối ưu ARM (NEON) có làm PQC khả thi trên SBC không. Đây là chỗ **liboqs bắt buộc** (OpenSSL native không có công tác NEON tường minh).

Task 3.1 — Có máy ARM

Bạn không có Pi vật lý → thuê ARM cloud: - **Oracle Cloud Always Free Ampere A1** (ARM Cortex-A, miễn phí vĩnh viễn) — khuyến nghị. - **AWS t4g** (Graviton2, free tier tới 31/12/2026). - PQC-LEO đo thật trên Raspberry Pi 4 (aarch64, Cortex-A72) — đúng phần cứng để bài gởi ý: <https://arxiv.org/pdf/2603.06149>

Task 3.2 — [!] Cái BÃY lớn nhất của WP3: build HAI cây liboqs

Đây là điểm tinh tế nhất cả đồ án. liboqs có cờ OQS_DIST_BUILD: - Trích tài liệu chính thức: When built for distribution, the library will run on any CPU... Function calls will be dispatched to micro-architecture optimized routines at run-time using CPU feature detection. - Link: <https://openquantumsafe.org/liboqs/api/doxygen/options-for-configuring-liboqs-builds.html>

Nghĩa là nếu OQS_DIST_BUILD=ON (mặc định), thư viện **tự động chọn NEON lúc chạy** dù bạn nghĩ đang đo “không NEON”. Để so *thật* ref-vs-NEON, phải build HAI cây tách biệt:

```
# CÂY REF (TẮT NEON): buộc dùng code C thuần
cmake -GNinja -S liboqs -B build-ref \
  -DCMAKE_INSTALL_PREFIX="$HOME/pqc/liboqs-ref" \
  -DBUILD_SHARED_LIBS=ON -DOQS_DIST_BUILD=OFF \
  -DOQS_USE_ARM_NEON_INSTRUCTIONS=OFF \
  -DOQS_OPT_TARGET=generic
ninja -C build-ref && ninja -C build-ref install

# CÂY NEON (BẬT NEON): tối ưu cho vi kiến trúc máy
cmake -GNinja -S liboqs -B build-neon \
  -DCMAKE_INSTALL_PREFIX="$HOME/pqc/liboqs-neon" \
  -DBUILD_SHARED_LIBS=ON -DOQS_DIST_BUILD=OFF \
  -DOQS_OPT_TARGET=auto
ninja -C build-neon && ninja -C build-neon install
```

- `OQS_OPT_TARGET=generic = -mcpu=cortex-a53` (an toàn, ít tối ưu); `auto = -mcpu=native`. Nguồn: <https://github.com/open-quantum-safe/liboqs/wiki/Customizing-liboqs/55cfed39e1027dd1d32170e6b91f557571b18d9e>
- `[!] OQS_DIST_BUILD=OFF` là *bắt buộc* để cờ NEON có hiệu lực — vì `OQS_OPT_TARGET` chỉ tác dụng khi `OQS_DIST_BUILD=OFF` (theo wiki). Quên tắt = cả hai cây đều dispatch runtime → đo ra giống nhau → kết luận sai.

Task 3.3 — Đếm chu kỳ chính xác trên ARM (PMU)

aarch64 không có `rdtsc` như x86. liboqs dùng PMU: - Trích: To get the more accurate cycle count, ARMv8's Performance Monitoring Unit (PMU) can be used via the `OQS_SPEED_USE_ARM_PMU` option. Access to the PMU from user mode must be enabled via a kernel module. - Link: <https://github.com/open-quantum-safe/liboqs/wiki/Platform-specific-notes-for-building-liboqs>

Thêm cờ khi cmake:

`-DOQS_SPEED_USE_ARM_PMU=ON`

- Cờ build thực tế trên Pi 4 (từ issue chính thức): `OQS_SPEED_USE_ARM_PMU, OQS_DIST_BUILD, OQS_OPT_TARGET=generic, compile -march=armv8-a+crypto -O3`. Link: <https://github.com/open-quantum-safe/liboqs/issues/1761>
- `[!] VPS cloud thường KHÔNG cho nạp kernel module (insmod)` → không bật được PMU. Lối thoát: đo bằng **wall-clock** (`CLOCK_MONOTONIC_RAW`, harness của bạn đã có) và ghi rõ trong **Limitations** rằng không lấy được cycle chính xác trên ARM thuê. Đề bài §13 cho phép điều này.
- Kernel module bật PMU user-mode: dự án `mupq/pqax` (`enable_ccr` cho `PMCCNTR_EL0`).

Task 3.4 — Kiểm chứng NEON thật sự bật

Chạy speed trên từng cây, so số: NEON phải nhanh hơn ref rõ rệt

`LD_LIBRARY_PATH="$HOME/pqc/liboqs-ref/lib" ./build-ref/tests/speed_kem ML-KEM-768`
`LD_LIBRARY_PATH="$HOME/pqc/liboqs-neon/lib" ./build-neon/tests/speed_kem ML-KEM-768`

- `[!] Kiểm tra ALGORITHMS.md của đúng tag liboqs` xem ML-DSA/ML-KEM có bản NEON aarch64 không — không phải thuật toán nào cũng có. Nếu một thuật toán không có NEON, ref $\approx$ neon là bình thường, ghi chú lại.

WP4 — TLS end-to-end (macrobenchmark, trả lời RQ3)

Mc tiêu: nhúng Kyber/Dilithium vào HTTPS server thật, đo handshake latency + throughput + bytes-on-wire. Đây là §7.6 “integrate into an application and measure end-to-end”. KHÔNG viết TLS từ đầu — cấu hình server dùng OpenSSL-PQC.

Task 4.1 — Tạo certificate ký bằng ML-DSA (nhúng Dilithium vào trục danh tính)

CÁCH A — Lệnh openssl req một dòng (có sẵn, native)

```
source scripts/setenv.sh
# Cert tự ký bằng ML-DSA-65 (Dilithium). Đây là "tích hợp Dilithium".
openssl req -x509 -newkey ML-DSA-65 -keyout srv.key -out srv.crt \
  -nodes -subj "/CN=localhost" -days 30
```

- Cú pháp này khớp guide thực hành PQC PKI (tạo CA ML-DSA-87 bằng `openssl req -x509 -newkey ML-DSA-87`): <https://community.f5.com/kb/technicalarticles/hands-on-quantum-safe-pki-a-practical-post-quantum-cryptography-implementation-g/344773>

- Và bài Medium hướng dẫn TLS 1.3 PQC: <https://medium.com/@truonghongcuong68/%EF%B8%8F-prepare-your-servers-tls-1-3-0bf61372c4c4>
- ML-DSA-65 → public key 1952 byte, **chữ ký 3309 byte** (nguồn học thuật): <https://arxiv.org/pdf/2604.04191>

CÁCH B — Tự dựng chuỗi CA → server cert (giống thực tế PKI)

```
# 1. CA gốc ký bằng ML-DSA-87
openssl req -x509 -newkey ML-DSA-87 -keyout ca.key -out ca.crt -nodes \
  -subj "/CN=PQC Test CA" -days 365 \
  -addext "basicConstraints=critical,CA:true" -addext "keyUsage=critical,key
# 2. Khóa + CSR cho server (ML-DSA-65)
openssl req -new -newkey ML-DSA-65 -keyout srv.key -out srv.csr -nodes -subj
# 3. CA ký cert server
openssl x509 -req -in srv.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
```

- [!] Nhớ hai trục độc lập của TLS 1.3: cert/CA = trục **chữ ký** (ML-DSA ở đây); -groups = trục **trao đổi khóa** (ML-KEM, task sau). Hai cái vuông góc, đừng lẫn.

Task 4.2 — Chạy HTTPS server nhỏ + chọn nhóm Kyber (nhúng ML-KEM vào trục khóa)

CÁCH A — openssl s_server (web server tí hon có sẵn — KHUYẾN NGHỊ)

```
# server: HTTPS nhỏ, trao đổi khóa ML-KEM-768
openssl s_server -accept 4433 -www -cert srv.crt -key srv.key -groups mlkem7
# client (của số khác): bắt tay thử
echo | openssl s_client -connect localhost:4433 -groups mlkem768 -brief
```

- s_server -www = server HTTP tối giản có sẵn trong OpenSSL, đúng nghĩa “small HTTPS server” của đề bài.
- RHEL lab dùng đúng s_server + s_time để đo handshake các nhóm KEM: <https://www.cryptomathic.com/ready-cryptography-with-openssl-3.5-on-rhel-9.6>
- [!] Lỗi bad record mac / decryption failed khi đổi -groups thường do client-server lệch nhóm hoặc lệch phiên bản OpenSSL. Đảm bảo cả hai dùng *cùng* openssl (cùng setenv.sh). Nguồn lỗi: <https://github.com/open-quantum-safe/oqs-provider/discussions/646>

CÁCH B — nginx biên dịch với OpenSSL-PQC (giống production)

Nếu muốn “HTTPS server” thực thụ:

```
# nginx.conf (rút gọn)
server {
    listen 4433 ssl;
    ssl_certificate      srv.crt;
    ssl_certificate_key  srv.key;
    ssl_protocols        TLSv1.3;
    ssl_ecdh_curve        X25519MLKEM768;    # nhóm hybrid PQC
}
```

- [!] Phải biên dịch nginx từ nguồn link tới OpenSSL 3.6 của bạn (nginx hệ thống link OpenSSL cũ, không PQC). FreeBSD Foundation mô tả rõ phải recompile nginx mỗi lần vá: <https://freebsdoundation.org/our-work/journal/browser-based-edition/embedded-2/implementing-a-quantum-safe-website/>
- Với đồ án, CÁCH A (s_server) đủ và gọn hơn nhiều; nginx chỉ làm nếu còn thời gian.

Task 4.3 — Đo ba chỉ số end-to-end (§9)

Handshake latency

```
# s_time: đếm số kết nối trong khoảng thời gian → suy ra latency trung bình
openssl s_time -connect localhost:4433 -new -time 10
```

Để lấy p95/p99: viết vòng lặp gọi s_client nhiều lần, ghi thời gian từng handshake vào CSV rồi tự thống kê.

Throughput (single + multi-thread — §7.4 đòi “test both single-threaded and multi-threaded server”)

```
# wrk hoặc ab tạo tải song song
wrk -t4 -c100 -d30s https://localhost:4433/
```

- Đề bài §8.2 liệt kê wrk/wrk2 hoặc ab.

Bytes-on-wire (network overhead THẬT, khác WP5)

```
# tshark bắt gói handshake, đo kích thước thật trên dây
tshark -i lo -f "tcp port 4433" -Y "tls.handshake" -V
```

- [!] Đây là chỗ phân biệt với WP5: WP5 đo kích thước lý thuyết (ML-KEM-768 ciphertext luôn 1088 byte — hằng số tra ra); WP4 đo byte thật chạy trên dây trong handshake (ClientHello to lên vì ML-KEM, Certificate to lên vì ML-DSA). Đừng gộp.

Task 4.4 — So ba cấu hình (chỉ đổi -groups + cert, giữ nguyên phần còn lại)

Cấu hình	KEM (-groups)	Cert (chữ ký)
Classical	X25519	ECDSA P-256
PQC thuần	mlkem768	ML-DSA-65
Hybrid	X25519MLKEM768	ML-DSA-65 (hoặc lai)

- Hybrid X25519MLKEM768 là mặc định Chrome/Firefox hiện tại: <https://www.cryptomathic.com/blog/quantum-ready-cryptography-with-openssl-3.5-on-rhel-9.6>
- [!] **Phương pháp công bằng:** chỉ thay đúng MỘT trục mỗi lần (đổi -groups mà giữ cert, hoặc đổi cert mà giữ -groups) → mới biết cái nào gây overhead. Đổi cả hai cùng lúc thì không tách được nguyên nhân (KEM hay chữ ký làm chậm).

WP5 — Tài nguyên tĩnh (kích thước, RAM, code size)

Tất cả đo tĩnh, một máy, không mạng. Ba nhóm.

Task 5.1 — Nhóm 1: Kích thước khóa / ciphertext / chữ ký (byte)

CÁCH A — Đọc thẳng từ liboqs (chính xác, là hằng số)

liboqs phơi sẵn các trường độ dài:

```
OQS_KEM *kem = OQS_KEM_new(OQS_KEM_alg_ml_kem_768);
printf("%zu %zu %zu\n", kem->length_public_key, // 1184
```

```
kem->length_secret_key,
kem->length_ciphertext); // 1088
```

- Để bài gọi ý đúng cách này (“read from liboqs: length_public_key, length_secret_key, length_ciphertext, length_signature”).

CÁCH B — Đo từ OpenSSL EVP (bench_evp đã làm)

bench_evp.cpp đã in size-pubkey-bytes, size-ciphertext-bytes, size-signature-bytes qua EVP_PKEY_get_octet_string_param / EVP_PKEY_get_size / encapsulate(NULL). Đã kiểm chứng RSA/EC trên sandbox. Đây là cách trong-luồng (cùng công cụ với phần đo thời gian). - Số tham chiếu để đối chiếu: ML-KEM-768 pk=1184/ct=1088; ML-DSA-65 pk=1952/sig=3309; ECDSA P-256 sig≈64-72.

Task 5.2 — Nhóm 2: Peak RSS (RAM cao nhất)

CÁCH A — /usr/bin/time -v (đúng công cụ để bài)

```
/usr/bin/time -v ./build/bench_evp mldsa 65 2>&1 | grep "Maximum resident se
```

- Để bài §7.4: “peak RSS via /usr/bin/time -v or pmap”. Script measure_memory.sh đã tự động hóa, đọc dòng “Maximum resident set size” (KB).
- [[]] /usr/bin/time (GNU time) KHÁC lệnh time builtin của shell. Phải dùng đường dẫn đầy đủ. Thiếu thì sudo apt install time.

CÁCH B — liboqs test_kem_mem / test_sig_mem

liboqs có sẵn chương trình đo bộ nhớ: - Trích README: test_kem_mem: Simple test harness for checking memory consumption of key encapsulation mechanisms; test_sig_mem: ... signature schemes. - Link: <https://github.com/open-quantum-safe/liboqs>

```
./build/tests/test_kem_mem ML-KEM-768 0
```

- Cách này tách stack/heap chi tiết hơn, nhưng chỉ cho thuật toán liboqs.

Task 5.3 — Nhóm 3: Code size (kích thước mã nhị phân)

CÁCH A — size (đúng công cụ để bài)

```
size "$HOME/pqc/openssl/lib64/libcrypto.so" # text/data/bss/total
size "$HOME/pqc/liboqs/lib/liboqs.so"
```

- Để bài §7.4: “code size via size utility, readelf -S”. Script measure_codesize.sh đã tự động hóa.

CÁCH B — readelf -S (chi tiết từng section)

```
readelf -S "$HOME/pqc/liboqs/lib/liboqs.so" | grep -E "text|data|bss"
```

- [[]] Để tách riêng code của một thuật toán, đo file object .o tương ứng trong cây build liboqs (vì libcrypto.so gộp tất cả, không tách được ML-DSA riêng). Ghi rõ trong báo cáo bạn đo ở mức nào (toàn thư viện vs từng .o).

WP6 — Năng lượng (ước lượng, phần phụ)

[!] Bạn thuê ARM từ xa → KHÔNG gắn được máy đo điện vật lý (Monsoon/INA219). Để bài §13 cho lối thoát: - Trích đề bài: “if not available, report only latency/throughput and estimate energy using CPU power models”. - Cách ước lượng: $\text{energy} \approx \text{thời_gian_thao_tác} \times \text{công_suất_CPU_danh_định}$ (TDP/số core). Đối chiếu với literature. - [!] BẮT BUỘC ghi vào **Limitations**: “không đo điện trực tiếp; ước lượng qua power model; con số chỉ mang tính tương đối.” Đây là phần “nếu có thể”, không bắt buộc cứng — đừng để nó chiếm thời gian của WP1-WP5.

WP7 — Thống kê + trả lời RQ + đóng gói

Task 7.1 — Phân tích thống kê (§7.5, ăn điểm Rigor 30%)

`analyze.py` đọc CSV thô → tính: - median-of-medians qua K batch (không phải mean đơn giản). - bootstrap CI (lấy mẫu lại nhiều lần) — đề bài ưu tiên “bootstrap or t-distribution”. - paired comparison (so cùng phần cứng để giảm phương sai). - [!] Loại warm-up trước khi tính (§7.5).

Task 7.2 — Biểu đồ trả lời ba RQ (§9 “Presentation”)

- latency vs parameter set (L1/L3/L5) — RQ1 compute.
- bytes vs algorithm (cột khóa/ct/sig) — RQ1 bandwidth → đây là biểu đồ “đắt giá” nhất, lộ ra chữ ký ML-DSA 3309B vs ECDSA 72B.
- throughput vs concurrency — RQ3.
- ref vs NEON trên ARM — RQ2.
- [!] So sánh phải **cùng mức bảo mật** (iso-security): ML-KEM-768 ↔ RSA-3072 ↔ P-384 (đều Level 3). So nhầm mức = kết luận sai.

Task 7.3 — Đóng gói tái lập (§11, ăn điểm Reproducibility 25%)

- `Dockerfile` cho x86 (đề bài §7.3, §11). Multi-arch cho ARM nếu được.
 - README ghi: phiên bản OpenSSL/liboqs (đã pin trong `versions.env`), lệnh chạy từng WP, governor CPU.
 - Nộp kèm: raw CSV, biểu đồ, bảng tổng hợp trade-off (§11 Deliverables).
-

Bảng tổng hợp: mỗi task → công cụ → link kiểm chứng

WP	Task	Cách A (có sẵn)	Cách B (tự thiết lập)	Link gốc
1	Build OpenSSL	<code>pqcbench rebuild-openssl.sh</code>	<code>./Configure --prefix=\$HOME</code>	openssl/INSTALL.md
1	Build liboqs	<code>oqs-provider fullbuild.sh</code>	<code>cmake</code> thủ công	openquantumsafe.org
2	Microbenchmark	<code>openssl speed / liboqs speed_*</code>	<code>bench_evp.cpp</code> (EVP)	liboqs README
3	NEON ARM	(không có sẵn)	hai cây ref/neon, <code>OQS_DIST_BUILD=OFF</code>	liboqs wiki
4	Cert ML-DSA	<code>openssl req -newkey ML-DSA-65</code>	chuỗi CA thủ công	F5 PKI guide
4	HTTPS server	<code>openssl s_server -groups mlkem768</code>	nginx + OpenSSL 3.6	RHEL lab
4	Đo hand-shake	<code>openssl s_time, wrk, tshark</code>	vòng lặp <code>s_client</code> tự ghi	OQS profiling

WP	Task	Cách A (có sẵn)	Cách B (tự thiết lập)	Link gốc
5	Kích thước	<code>liboqs length_*</code>	<code>bench_evp</code> EVP sizes	đề bài §9
5	Peak RSS	<code>/usr/bin/time -v</code>	<code>liboqs test_*_mem</code>	đề bài §7.4
5	Code size	<code>size</code>	<code>readelf -S</code>	đề bài §7.4

Danh sách link kiểm chứng (gom lại để tiện check)

1. OpenSSL 3.5 PQC (FIPS 203/204/205): <https://openssl-library.org/post/2025-02-04-release-announcement-3.5/>
2. OpenSSL 3.5 final/LTS: <https://openssl-foundation.org/post/2025-04-08-openssl-35-final-release/>
3. OpenSSL INSTALL.md: <https://github.com/openssl/openssl/blob/master/INSTALL.md>
4. pqcbench (lelegard): <https://github.com/lelegard/pqcbench>
5. liboqs: <https://github.com/open-quantum-safe/liboqs>
6. liboqs getting-started: <https://openquantumsafe.org/liboqs/getting-started.html>
7. liboqs build options: <https://openquantumsafe.org/liboqs/api/doxygen/options-for-configuring-liboqs-builds.html>
8. liboqs ARM PMU notes: <https://github.com/open-quantum-safe/liboqs/wiki/Platform-specific-notes-for-building-liboqs>
9. liboqs customizing (OPT_TARGET): <https://github.com/open-quantum-safe/liboqs/wiki/Customizing-liboqs/55cfed39e1027dd1d32170e6b91f557571b18d9e>
10. liboqs Pi4 build flags (issue): <https://github.com/open-quantum-safe/liboqs/issues/1761>
11. oqs-provider: <https://github.com/open-quantum-safe/oqs-provider>
12. OQS profiling (openssl speed/s_time): <https://github.com/open-quantum-safe/profiling>
13. PQC-LEO framework: https://github.com/crt26/PQC-LEO/blob/main/docs/performance_results/performance.md
14. PQC-LEO paper (Pi4 Cortex-A72): <https://arxiv.org/pdf/2603.06149>
15. RHEL 9.6 PQC lab (s_server/s_time): <https://www.cryptomathic.com/blog/quantum-ready-cryptography-with-openssl-3.5-on-rhel-9.6>
16. F5 PQC PKI hands-on: <https://community.f5.com/kb/technicalarticles/hands-on-quantum-safe-pki-a-practical-post-quantum-cryptography-implementation-g/344773>
17. TLS 1.3 PQC (cert ML-DSA): <https://medium.com/@truonghongcuong68/%EF%B8%8F-prepare-your-servers-tls-1-3-0bf61372c4c4>
18. FreeBSD quantum-safe nginx: <https://freebsdoundation.org/our-work/journal/browser-based-edition/embedded-2/implementing-a-quantum-safe-website/>
19. ML-DSA-65 sizes (3309B sig): <https://arxiv.org/pdf/2604.04191>
20. oqs-provider bad-record-mac debug: <https://github.com/open-quantum-safe/oqs-provider/discussions/646>